

GNN Trajectory prediction

Group 33

Project in one sentence

Given the last 2 seconds of a car's motion (+ its surroundings), predict its next 6 seconds, then measure when the surroundings actually help.

What data are we using?

NuScenes gives driving scenes with every car, pedestrian and cyclist tracked at 2 Hz. For one **target** agent at one moment,

We're using the **v1.0 trainval** to which we loaded into our workspace and it contains:

- 850 scenes
- 34,139 samples
- 1,166,187 sample annotations
- 64,384 tracked instances

It's also relevant to define what the dataset terms are:

- A **scene** is one driving sequence
- A **sample** is one timestamp within that scene
- A **sample annotation** is one object at that one timestamp
- An **instance** is one physical object tracked over time: **instance_token**

So an instance can be present in many samples and scenes for example

We have **instance_token** = car A

- car A at t_0 → annotation
- car A at t_1 → annotation
- car A at t_2 → annotation

For each prediction sample we have:

- Target **past trajectory**: **5 x 2**
- Target **future trajectory**: **12 x 2**
- A variable number of neighbors
- Each neighbor **past neighbor**: **5 x 2**
- Each neighbor category
- All trajectories already transformed into the **target local frame**

How to turn one prediction sample into one graph?

The cleanest way to think is in four parts:

$$[x, edgeIndex, edgeAttribute, y]$$

What does each node represent?

Each **agent becomes a node** so one sample might contain:

- node 0 = target
- node 1 = car
- node 2 = pedestrian
- node 3 = truck
- node 4 = bicycle

Every element is a node. So for this case we have $N = 5$ nodes. So depending in the sample we're analyzing N can change and this is exactly what the GNN are designed for.

Node features x : what information does each agent carry?

Each agent has a **2s past** with **5 positions history**:

$$[(x_{-4}, y_{-4}), \dots (x_0, y_0)]$$

by flattening we have 2 (x, y) x 5 positions = 10 numbers.

$$[x_{-4}, y_{-4}, x_{-3}, y_{-3}, \dots x_0, y_0]$$

So first info that the node carries is position. We can also add agent **category as**:

- Car
- Large vehicle
- Two wheeler
- Pedestrian
- Other vehicles

Which can be represented as **one-hot vector** for example:

Car = [1, 0, 0, 0, 0] or Pedestrian = [0, 0, 0, 1, 0]

So far the node agent feature is represented by a 15 element vector:

$$x_i = [trajectory(10), type(5)]$$

So the total number of features is $x \in \mathbb{R}^{N \times 15}$ N being the number of nodes (agents) in that sample.

To simplify we assume **node 0 to always be the target vehicle.**

Edge: Who talks with who?

Now we start building the actual graph by connecting each node. Each node connection creates a **bi-directional edge**, and essentially the information from one node can influence another node and vice-versa

To connect different nodes agents, we can use a **radius graph neighbor**. So we connect two agents if they're sufficiently close to each other

$$(i, j) \in E \Leftrightarrow ||p_i - p_j|| < r$$

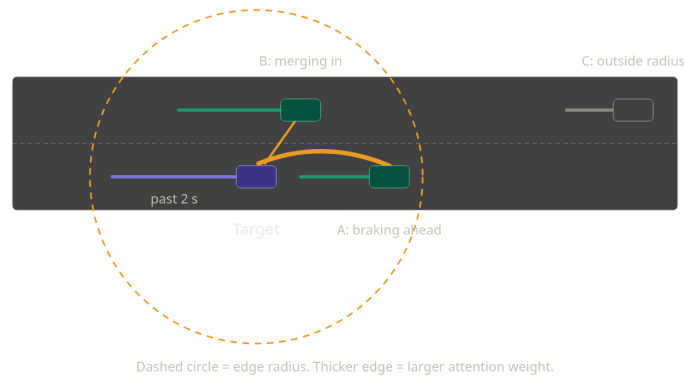
Having an edge we need to describe the relationship between 2 connected agents i and j . The simplest useful edge feature is relative position and its distance d :

$$\Delta x_{ij} = x_j - x_i \text{ and } \Delta y_{ij} = y_j - y_i \rightarrow d = \sqrt{\Delta x_{ij}^2 + \Delta y_{ij}^2}$$

So an edge would carry for example:

$$e_{ij} = [\Delta x, \Delta y, d] \in R^{E \times 3}$$

And later we could the relative velocities Δv to the edge



Output prediction $\hat{y}$:

The only label is the target's future trajectory not the neighbors:

$$\hat{y} = [(x_1, y_1), \dots (x_{12}, y_{12})] \in R^{12 \times 2}$$

Shapes Summarized for an example case)

$x = [4 \text{ nodes}, 15 \text{ features (10 past trajectories + 5 one-hot encodings)}]$

edge_index = [2, 8]

edge_attr = [8, 3] (8 bidirectional edges) x (delta x, delta y, d)

y = [12, 2] (12 timestamps (6s future data starting at t0) x (x, y)

Compute Loss and metrics

The model does **not** see the future as input. We only use it as the ground-truth target to train and evaluate against.

So when we computed “future displacement,” we were not predicting anything yet. We were just measuring the known ground-truth trajectory that already exists in nuScenes.

So using the “known future” we can train the network with a **Average Displacement Error Loss (ADE)**

$$L = \frac{1}{12} \sum_{t=1}^{12} \|\hat{p}_t - p_t\|_2$$

Target-local coordinates

During preprocessing we expressed everything in target’s frame such that $p_0^{target} = (0, 0)$ and the target’s future or is heading in **+ x** direction on the plot.

This is good for learning because it allows the network to simply focus on local coordinates and not global. This means, the network won’t be worried in which geo-position is the event happening i.e in Boston or Singapore.

Upgrades

The main upgrades we would look forward to implement once the base-version is done is to add:

1. When does relying on surroundings vehicles become vulnerable?

030728xyr@gmail.com : <https://github.com/zqzqz/AdvTrajectoryPrediction>

2. Multi-modal K = 6

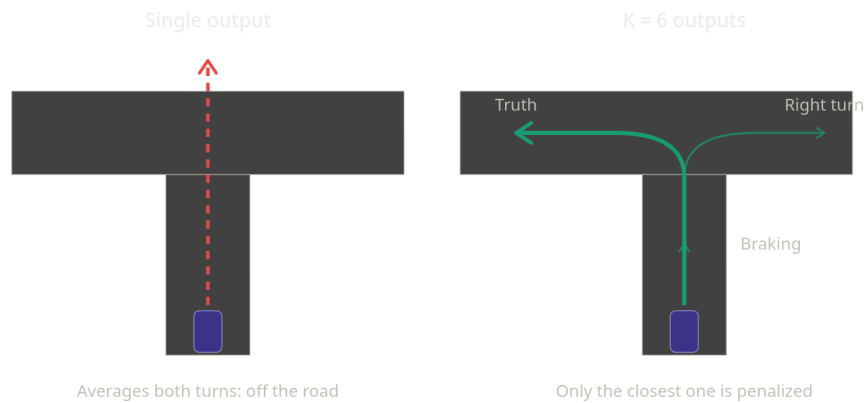
Will only change the output so that instead of having $\hat{y} = [12, 2]$ it will have $\hat{y} = [6, 12, 2]$ there will be 6 more **candidates instead of a single one.**

3. Attention

Instead of aggregating neighbors equally we give some weights. Where important neighbor receive larger α_i

4. Map and lane context

Adding lane information as additional nodes having agent nodes + lane nodes.



Resources

https://www.nuscenes.org/tutorials/nuscenes_tutorial.html

<https://foxglove.dev/blog/visualizing-nuscenes-data-with-foxglove-studio>

<https://arxiv.org/pdf/2005.04259>: VectorNet

<https://arxiv.org/pdf/2201.05057>: Adversarial Robustness of Trajectory Prediction

Example sample from a NuScenes token

The target vehicle in this case is the one shown in the CAM_BACK view.

